

Portafolio

Caso de estudio: Creación de un producto digital serverless

Nombre y apellido: Nathaly Bravo Valenzuela

Fecha: Septiembre de 2026

1. Presentación

Mi nombre es Nathaly Bravo Valenzuela, estudiante de un programa intensivo en tecnología, con interés en la arquitectura de soluciones en la nube. Diseñé y documenté seis proyectos en AWS: desde un monolito escalable hasta microservicios, arquitecturas serverless y controles de seguridad para una fintech. Me caracterizo por probar lo que construyo (pruebas de carga, alarmas y fallas simuladas), documentar con claridad y aprender de forma continua. Busco mi primera experiencia como Talento Junior Tech para aportar en proyectos cloud y de transformación digital.

Mis datos de contacto son:

Nombre	Nathaly Bravo Valenzuela
Correo electrónico	natha.bravova@gmail.com
LinkedIn	https://www.linkedin.com/in/nathalybravo/
GitHub (opcional)	https://github.com/nathabra
Ciudad y país	Santiago, Chile
CV	https://drive.google.com/file/d/1_wlHkG5of7lvYlks7usxDn5_JVnct-nl/view?usp=drive_link

2. Portafolio técnico

2.1 Proyectos trabajados

Proyecto	Qué hice	Tecnologías
Portafolio 3 · Nube Sólida	Diseñé una arquitectura conceptual cliente-servidor híbrida, asignando el modelo de servicio adecuado (PaaS, FaaS, IaaS) a cada componente. Incluí API Gateway como entrada única, mensajería asíncrona, cifrado AES-256 y TLS 1.3, Zero Trust y pruebas de Chaos Engineering.	Nube híbrida, PaaS, FaaS, API Gateway, SQS
Portafolio 5 · Cloud básicas e integración híbrida	Diseñé una arquitectura que combina nube pública y privada: S3 con ciclo de vida hacia Glacier, Direct Connect y VPC, Auto Scaling en dos zonas de disponibilidad, CloudFront y mensajería SNS/SQS. Estimé costos (~USD 486–606 al mes, optimizables a ~USD 330–410).	S3, Glacier, Direct Connect, VPC, EC2, ELB, CloudFront, SNS, SQS
Módulo 6 · Monolito escalable en AWS	Migré un monolito a AWS con alta disponibilidad: Launch Template, ALB y Auto Scaling (1–2 instancias, escalado al 60 % de CPU), dos Security Groups y DynamoDB. Validé la alarma de CloudWatch (80 % de CPU) con la herramienta stress.	EC2, ALB, ASG, Security Groups, DynamoDB, CloudWatch
Portafolio 7 · MicroPay	Pasé de un monolito a dos microservicios en Node.js (usuarios y pagos) con una base de datos por servicio, contenedores Docker en ECR y orquestación en ECS Fargate con autoescalado (2 a 4 tareas al 70 % de CPU). Probé la carga con la herramienta hey.	Node.js, Docker, ECR, ECS Fargate, ALB, API Gateway, DynamoDB
Portafolio 8 · Serverless (caso de estudio destacado)	Construí una arquitectura sin servidores con sitio estático en S3, funciones Lambda en Python, API Gateway y DynamoDB, con costo cero sin tráfico. En el Portafolio 10 la evolucioné a una aplicación completa (ver sección 2.4).	S3, Lambda (Python), API Gateway, DynamoDB, CloudWatch
Portafolio 9 · Blue Wave (Cloud Secure)	Implementé controles de seguridad y cumplimiento para una fintech: mínimo privilegio con IAM, bloqueo de acceso público y cifrado en S3, auditoría con CloudTrail, reglas de AWS Config y alarma de CPU con aviso por SNS.	IAM, S3, CloudTrail, AWS Config, CloudWatch, SNS

2.2 Caso de estudio: Pedidos Serverless

Pedidos Serverless: una aplicación sin servidores, segura y observable en AWS.

1. Descripción del proyecto

Diseñé e implementé en AWS Academy una aplicación web de gestión de pedidos 100 % serverless. La persona usuaria abre un sitio estático alojado en Amazon S3, registra pedidos desde un formulario y consulta el listado. El navegador se comunica con una API HTTP (Amazon API Gateway) que ejecuta una función escrita en Python en AWS Lambda, la cual guarda y lee los datos en Amazon DynamoDB. La solución incluye monitoreo con Amazon CloudWatch y alertas por correo con Amazon SNS.

2. Desafío principal

Lograr que cinco servicios independientes funcionaran como un solo sistema sin administrar servidores, con costo casi nulo en reposo y con controles básicos de seguridad y observabilidad, trabajando dentro de las restricciones del Learner Lab (una sola región permitida y un único rol preexistente, LabRole).

- Entender cómo API Gateway entrega los datos a Lambda y qué formato de respuesta espera.

- Resolver el bloqueo CORS del navegador entre el sitio (S3) y la API (otro dominio).
- Comprobar que las alarmas se disparan de verdad, no solo que están configuradas.

Diagrama de arquitectura · Pedidos Serverless

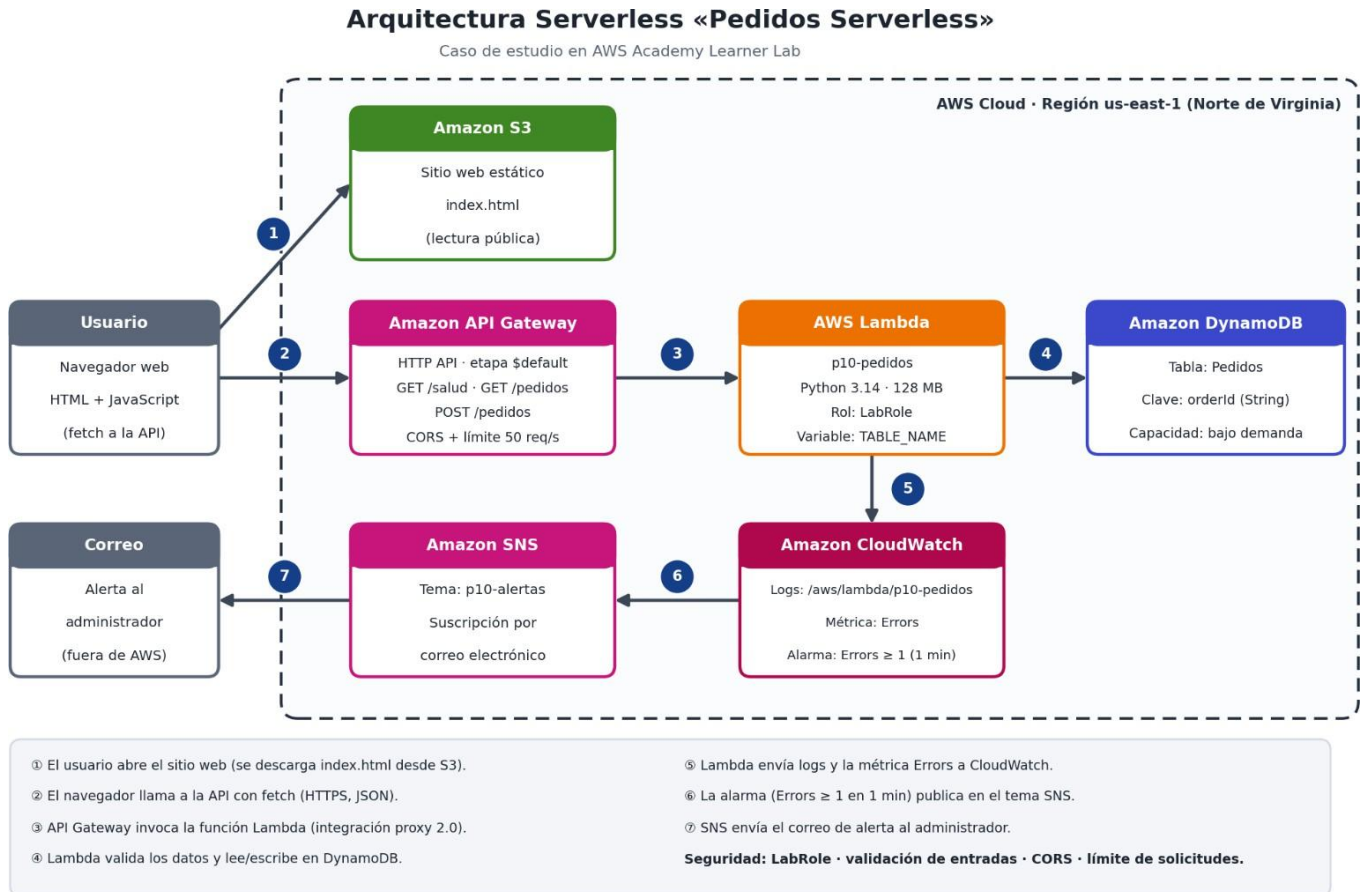


Figura 1. Arquitectura del caso de estudio en AWS (región us-east-1). Los números indican el flujo de una solicitud y de una alerta.

3. Solución propuesta

La solución permite implementar una arquitectura Serverless en AWS utilizando Amazon S3, API Gateway, AWS Lambda y DynamoDB. Esta solución permite ejecutar la aplicación sin administrar servidores, delegando al proveedor cloud la infraestructura, escalabilidad y disponibilidad.

Utilizar API Gateway como punto de entrada, Lambda para ejecutar la lógica de negocio, S3 para alojar contenido estático y DynamoDB para gestionar los datos. Complementar la arquitectura con IAM/LabRole para controlar permisos y CloudWatch para monitorear el funcionamiento.

De esta manera, obtener una solución escalable, modular, administrable y orientada al pago por uso, permitiendo concentrar el desarrollo en la lógica de negocio mientras AWS gestiona la infraestructura subyacente.

4. Herramientas técnicas utilizadas

- **AWS Lambda** (Python 3.14, biblioteca boto3) para la lógica de negocio.
- **Amazon API Gateway** (HTTP API) como punto de entrada único, con CORS y límite de solicitudes.
- **Amazon DynamoDB** (tabla Pedidos, capacidad bajo demanda) para la persistencia NoSQL.
- **Amazon S3** (alojamiento de sitio web estático) con HTML y JavaScript.
- **Amazon CloudWatch** (registros, métricas y alarma) y **Amazon SNS** (notificación por correo).
- **AWS Academy Learner Lab** (rol LabRole) y **Google Sites** para publicar el portafolio.

5. Principales aprendizajes alcanzados

- **Integración de servicios:** cada servicio cumple una función específica y se comunica por eventos y permisos; entender el flujo completo es más importante que dominar un solo servicio.
- **Lambda y API Gateway:** el evento de entrada trae el método y la ruta, y la función debe devolver statusCode, headers y body.
- **CORS:** el navegador bloquea llamadas entre dominios distintos si la API no las autoriza; se resuelve configurando orígenes, métodos y cabeceras permitidos.
- **DynamoDB:** la clave de partición define cómo se identifican los datos, y en Python los números llegan como Decimal, por lo que hay que convertirlos.
- **Arranque en frío:** la primera llamada tras un período sin uso es más lenta que las siguientes; medirlo permite explicar la latencia con datos.
- **Observabilidad:** una alarma solo es confiable si se prueba; la falla simulada demostró que el aviso llega por correo.
- **Límites del laboratorio:** en el Learner Lab solo se puede usar LabRole; en producción correspondería crear un rol de mínimo privilegio.

6. Métricas de impacto logradas

Métrica	Cómo se midió	Resultado
Tiempo total de implementación	Cronómetro desde crear la tabla hasta la prueba de punta a punta.	60 min
Servidores administrados	Conteo de servidores o instancias en la arquitectura.	0 (diseño 100 % serverless)
Latencia con arranque en frío	Primera llamada tras 15 minutos sin uso; se compara con Init Duration en CloudWatch Logs.	11 ms
Duración facturada y memoria usada por invocación	Línea REPORT en CloudWatch Logs.	1.97 ms / 105 MB de 128 MB
Tiempo de detección de una falla	Hora de la falla simulada frente a la hora del correo de SNS.	+5 min
Costo sin tráfico	Diseño serverless: sin cómputo ni capacidad aprovisionada.	≈ USD 0 (solo KB de almacenamiento en S3)
Costo estimado con 100.000 solicitudes al mes	Calculadora de precios de AWS.	≈ USD 0.02

7. Habilidades técnicas aplicadas

Python y biblioteca boto3 · diseño de API REST (métodos, rutas y códigos de estado HTTP) · JSON · modelado NoSQL en DynamoDB · HTML y JavaScript (fetch) · CORS · conceptos de IAM y mínimo privilegio · validación de entradas · monitoreo con CloudWatch y SNS · pruebas de rendimiento y de falla · estimación de costos · documentación técnica con diagramas.

8. ¿Por qué elegí este proyecto?

Porque integra lo aprendido en mis seis proyectos personales: el modelo NoSQL de **Monolito escalable en AWS** y DE **MicroPay**, el punto de entrada único con API Gateway de **MicroPay**, la seguridad y el monitoreo del **Blue Wave** y la optimización de costos de **Cloud básicas e integración híbrida**. Además, responde directamente a la vacante: aplica buenas prácticas de diseño técnico en la nube, propone una solución a una necesidad concreta y documenta el proceso con evidencias. Finalmente, muestra mi crecimiento: pasé de leer datos con una función a entregar una aplicación completa, medible y con alertas.